



Learning to Race with PPO

How Policy Network Size Affects

Performance and Training Time

Master's Degree in Computer Science

Yagiz Tansu (1234567)

June 18, 2026



UNIVERSITÀ
DEGLI STUDI
DI MILANO



Table of Contents

1 Introduction

► Introduction

► Environment

► Method & Experiments

► Results

► Conclusion



Problem & Goal

1 Introduction

Task

Drive a car around a **2D closed racetrack** in **minimum time**.

Approach

- No hand-coded racing line
- Learn from rewards (RL)
- Continuous throttle + steering

Stack

Custom **Gymnasium** env

PPO (Stable-Baselines3)

Python + NumPy + PyTorch

Research question

Does a **bigger policy network** improve **(1)** final lap quality, **(2)** episodes to learn, and **(3)** wall-clock training time?



Project Pipeline

1 Introduction

Step 1 — Build the simulation



Step 2 — Train

train.py
PPO · Gaussian policy
3 networks × 4 seeds
1M steps · random track / ep

Step 3 — Evaluate & plot

evaluate.py + plots.py
deterministic laps
reference circuit + held-out
mean ± std across seeds



Table of Contents

2 Environment

► Introduction

► **Environment**

► Method & Experiments

► Results

► Conclusion



Track Generation

2 Environment

Pipeline (`track.py`)

Control points — 8–12 (x, y) waypoints (closed loop)

Periodic spline — smooth centreline, start = finish

Arc-length resample — uniform spacing every 1 m

Precompute — tangent $\hat{t}$, signed curvature κ

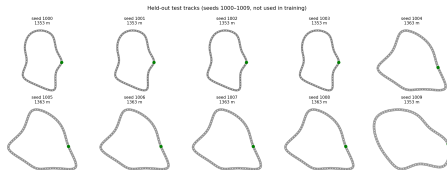
Stored per sample

Centreline point (x, y)

Cumulative arc length s

Unit tangent (heading of the road)

$\kappa > 0$: left turn $\kappa < 0$: right turn



Three track modes

Train: random pool (seeds 0–49)

Compare: reference circuit (`track()`)

Generalise: held-out (seeds 1000–1009)



Vehicle Dynamics

2 Environment

Kinematic bicycle model (`car.py`)

Rear-axle reference, centre of mass at mid-wheelbase.

Forward Euler integration, step $dt = 0.05$ s (20 Hz).

Update equations

$$\beta = \arctan(\tan(\delta) \cdot \frac{l_r}{L})$$

$$x \leftarrow x + v \cos(\psi + \beta) dt$$

$$y \leftarrow y + v \sin(\psi + \beta) dt$$

$$\psi \leftarrow \psi + \frac{v}{l_r} \sin(\beta) dt$$

$$v \leftarrow \text{clip}(v + a dt, 0, v_{\max})$$

Physical limits

Parameter	Value
Max speed $v_{\max}$	30 m/s
Max accel. $a_{\max}$	8 m/s ²
Max steer $\delta_{\max}$	0.45 rad ($\approx 26^\circ$)
Wheelbase L	3.0 m
Time step dt	0.05 s

Why this model?

Simple enough for RL, yet captures steering–speed coupling for cornering.



Observation Space — Frenet Coordinates

2 Environment

Key idea

The agent answers: “Where am I on the road?” — not “Where am I on the map?”

Frenet frame (intuition)

d — lateral offset from centreline

$d > 0$ left $d < 0$ right

θ_e — heading error vs. road tangent

0 = aligned with the track

Lookahead κ — signed curvature ahead
positive = left turn coming

Observation vector — 14 numbers

Index	Feature	Meaning
0	$d / 6 \text{ m}$	distance from centre
1-2	$\sin \theta_e, \cos \theta_e$	heading error
3	$v / 30 \text{ m/s}$	normalised speed
4-13	$\kappa \times 20$	curvature at 10 lookaheads 5, 10, ..., 110 m ahead

(x, y) and progress s are **not** in the state
⇒ better generalisation across tracks

Why $\sin / \cos$? Angle is continuous (no jump at $\pm\pi$).



Reward & Episode Rules

2 Environment

Per-step reward

$$r_t = \Delta s - 0.01$$

Reward forward progress; small penalty discourages stopping.

Episode end

Event	Effect
$ d > 6$ m	-80, terminated
Lap done	+100, terminated
2000 steps	truncated

Gymnasium `terminated` vs `truncated` is respected for correct PPO bootstrapping.



Table of Contents

3 Method & Experiments

- ▶ Introduction
- ▶ Environment
- ▶ Method & Experiments
- ▶ Results
- ▶ Conclusion



Why PPO?

3 Method & Experiments

What is it?

PPO (Proximal Policy Optimization) learns a **policy** $\pi(a | s)$: a neural network that picks throttle and steering from the current observation.

How it works (simple)

- Agent tries actions in the environment
- Good outcomes → reinforce those actions
- Bad outcomes → discourage them
- PPO updates in small, safe steps (stable training)

In this project

- Continuous control** — throttle + steering fit naturally
- Gaussian policy** — outputs mean action + exploration noise
- On-policy** — learns from its own recent driving data
- Stable-Baselines3** — reliable PPO implementation



Experiment Grid

3 Method & Experiments

Training setup

Shared MLP **actor-critic** (same architecture)

8 parallel environments (SubprocVecEnv)

1M steps per run

4 seeds per network size

Random track each episode (pool 0-49)

Identical PPO hyperparameters

What we change

Only the **hidden layer sizes** — everything else stays the same.

Network sizes

Label	Architecture	Params
Tiny	[16]	~400
Small	[64, 64]	~9k
Large	[256, 256]	~132k

Evaluation (two stages)

Reference circuit — all 12 runs (fair arch. comparison)

Held-out tracks — winning checkpoint only (generalisation)



Table of Contents

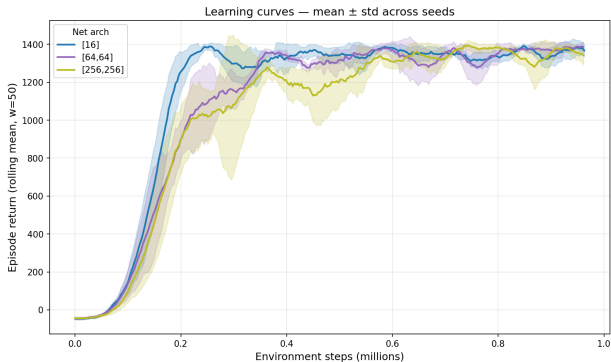
4 Results

- ▶ Introduction
- ▶ Environment
- ▶ Method & Experiments
- ▶ **Results**
- ▶ Conclusion



Learning Curves

4 Results



How to read

x-axis: cumulative env. steps

y-axis: return (50-ep. rolling mean)

shaded band: std across 4 seeds

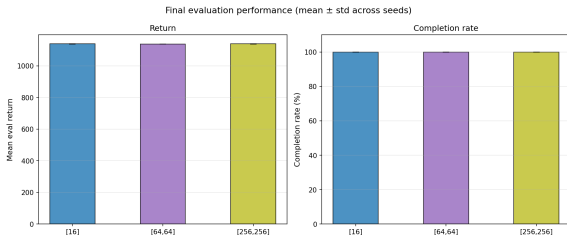
Results

Random track each episode $\Rightarrow$ noisier curves than fixed-track training.
All architectures still reach ~ 1100 – 1140 return; final performance is similar.



Final Performance (Reference Circuit)

4 Results



After training (4 seeds, ref. circuit)

Net	Return	Lap (s)
[16]	1139	36.0
[64,64]	1139	35.9
[256,256]	1139	36.0

100% completion on every seed.

Lap times differ by < 0.1 s.

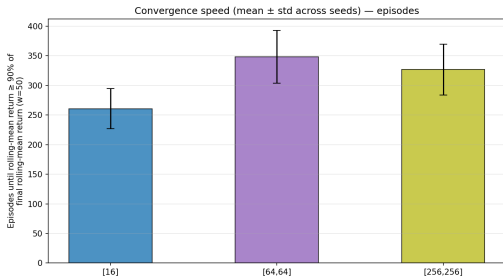
Winner: [16] (mean completion, return, lap time).



Training Cost: Episodes vs Wall-Clock

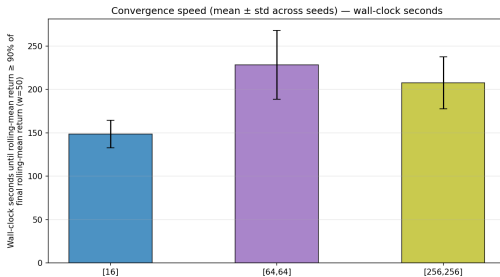
4 Results

Episodes to converge



Best: [16] (261 ep)

Wall-clock to converge



Best: [16] (149 s)

Key insight

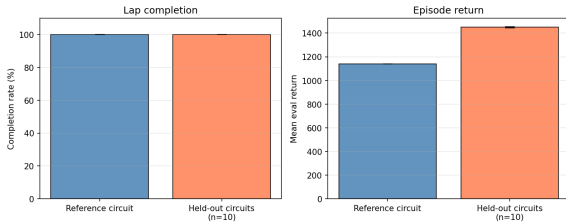
With random-track training, [16] wins on **both** episodes and wall-clock. [64,64] needs the most episodes (348) and the longest real time (228 s).



Generalisation to Held-Out Tracks

4 Results

Generalization — arch16_seed3_random
(trained on random circuits, seeds 0–49)



Checkpoint

arch16_seed3_random
(best seed of winning arch. [16])

Test set

10 procedural tracks (seeds 1000–1009)
never seen in training
length ~1350–1360 m

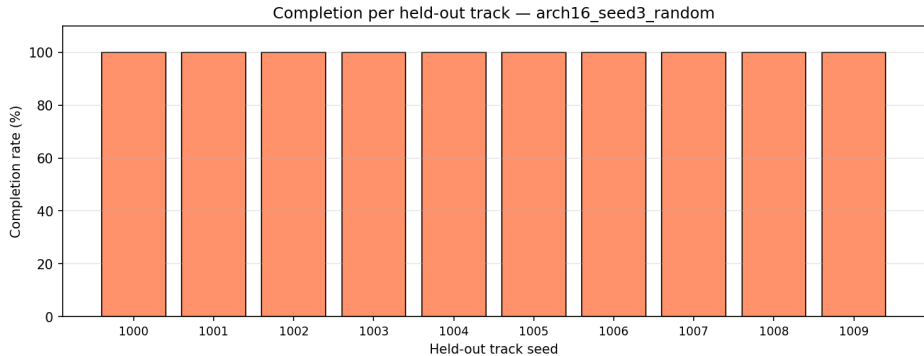
Result

100% lap completion
on reference circuit **and** all held-out tracks.



Generalisation per Held-Out Track

4 Results



One bar per held-out seed (1000–1009). All tracks completed — random-track training transfers to unseen layouts.



Table of Contents

5 Conclusion

► Introduction

► Environment

► Method & Experiments

► Results

► Conclusion



Takeaways

5 Conclusion

1. **Reference performance** — All three networks complete the lap; returns ~ 1139 ; lap times within 0.1 s.
2. **Winning architecture** — [16] on the reference benchmark (completion, return, lap time).
3. **Training cost** — [16] fewest episodes (261) **and** shortest wall-clock (149 s).
4. **Generalisation** — arch16_seed3_random: 100% completion on 10 held-out tracks.

Main message

Bigger is not always better.

Random-track training + [16] = best efficiency and strong generalisation.



Learning to Race with PPO *Thank you for*

listening!
Any questions?